

Αναλυτική περιγραφή Εφαρμογής, Αρχιτεκτονικής και τεχνολογιών

Σημαντική σημείωση. Το κομμάτι με το recommendation που αφορά τα ratings δεν έχει προλάβει να ολοκληρωθεί οπότε να μην ληφθεί υποψίνα απλώς είναι για μελλοντική κυρίως επέκταση παρόλα αυτά θα εξηγηθεί η βασική ιδέα. Επίσης καθυστερεί υπερβολικά να γίνουν import τα ratings γιατί είναι εκατομμύρια εγγραφές από ένα dataset που βρήκα.

Αρχική συντομή περιγραφή εφαρμογής

Πλήρης περιγραφή εφαρμογής Προκειται για μια full-stack εφαρμογή αναζήτησης ταινιών με βάση την περιγραφή. Είναι φτιαγμένη με RBAC αρχιτεκτονική που σημαίνει μπορεί να συνδεθεί είτε ως admin με διαφορετικό template είτε ως απλός χρήστης. Ο χρήστης μπορεί να αναζητά ταινίες τόσο με βάση τον τίτλο keywords όσο και με βάση μια περιγραφή της πλοκής/σεναρίου ταινίας που αναζητάει. Μπορεί επίσης να προσθέτει ταινίες στα αγαπημένα του και να συνομιλεί με ένα GenAI σύστημα που απαντά σε ερωτήσεις σχετικές με ταινίες. Έχει ένα ανολοκληρωτό προς το παρών recommendation system που βασίζεται στην τεχνική item-based collaborative filtering με δυνατότητα προέκτασης στο μέλλον που αφορά το content semantic recommendation βάσει ενδιαφερόντων.

Ο βασικός στόχος της εφαρμογής είναι να ξεπεράσει τους περιορισμούς μιας απλής keyword αναζήτησης. Αντί να βασίζεται μόνο σε τίτλους ή genres, χρησιμοποιώ vectorstore με embeddings των plots, summaries και LLM-generated semantic queries για κάθε ταινία προσομοιάζοντας ερωτήσεις του χρήστη, ώστε να καταλαβαίνει καλύτερα το νόημα του αιτήματος του χρήστη. Έτσι μπορεί να επιστρέφει ταινίες που ταιριάζουν θεματικά, συναισθηματικά ή αφηγηματικά με αυτό που ζητά ο χρήστης.

Η εφαρμογή συνδυάζει angular web αρχιτεκτονική με python fastapi για backend συνδυάζοντας σχεσιακή βάση δεδομένων mysql μέσω SQLAlchemy pydantic καθώς και διανυσματική chroma database, REST API και frontend UI, μαζί με τεχνικές Generative AI όπως semantic search, Retrieval-Augmented Generation και graph workflow.

Παράδειγμα ερωτήματος:

- "Ταινία με διαστημικό ταξίδι όπου ο χρόνος κυλάει διαφορετικά" (Plot-based query),
- "Movies with neon visuals and synth soundtrack"(Cinematic style query)
- "Sci-fi detective movie with philosophical themes" (πχ βάζω filter genre)
- "Ταινίες που προκαλούν anxiety και tension συνεχώς", (Psychological/emotional)
- "Χαρούμενες ταινίες για να δω όταν είμαι πεσμένος", (Vibe Queries)
- "Σκοτεινές και ανατριχιαστικές ταινίες ψυχολογικού τρόμου"

Δεδομένα που χρησιμοποιήθηκαν για την εφαρμογή και τα docs

Για τα σενάρια και τις περιγραφές των ταινιών βρήκα το wiki_movie_plots_deduped.csv το οποίο περιέχει ταινίες με την περιγραφή του σεναρίου που υπάρχει στο wikipedia με ονόμα στηλης plot με στηλες Release Year, Title, Origin/Ethnicity, Director, Cast, Genre, Wiki Page, Plot. Από τις οποίες για χαριν απλοτητας χρησιμοποιήθηκαν μόνο οι εξής: Year, Title, Director, Genre, Plot. Επίσης για το recommendation χρησιμοποιήσα ένα dataset από το movieLens <https://grouplens.org/datasets/movielens/10m/> γνωστό ως MovieLens 10M

Dataset με τίτλο ml-10m.zip το οποίο περιέχει μεν αξιολογήσεις αλλά δεν έχει τα σενάρια και τις περιγραφές των ταινιών. Έχει 2 datasets, ένα με ταινίες κι ένα με τις αξιολογήσεις των χρηστών σε αυτές με τα IDs. movies.dat και ratings.dat. Έτσι τα συνδίασα με το dataset wiki_movie_plots_deduped.csv κάνοντας merge με βάση τον τίτλο και το έτος για να χρησιμοποιήσω αξιολογήσεις για recommendation πάνω στα plots του wiki_movie_plots_deduped.csv. Το αρνητικό με αυτό το dataset σχετικά είναι ότι κάθε σενάριο ταινίας επειδή είναι από το wiki είναι αρκετά μεγάλο και έχει ολοκληρές παραγράφους ενώ η αρχική μου ιδέα ήταν να βρω μικρά σενάρια που υπάρχουν σε διάφορα sites 2-3 προτάσεων ώστε για χαριν απλότητας να κάνω μοναχά 1 διανυσμα για κάθε ταινία. Αυτό ήταν κάτι που με δυσκόλεψε καθώς ήταν αρκετές οι ταινίες και θα χρειαζόταν κάποιο chunk που θα καθυστέρουσε πολύ την εκτέλεση και την δημιουργία της βάσης τον περιερισμό να τα ξαναφτιάχνω και διορθώνω αλλά το έσπασα απλά σε παραγράφους ως chunk. Επίσης χρησιμοποίησα ένα LLM ώστε για κάθε ταινία να κάνει 2 ερωτήσεις παρομοιάζοντας με την ερώτηση που θα έκανε ένας χρήστης ώστε να του φέρει την συγκεκριμένη ταινία. Επίσης χρησιμοποίησα πάλι ένα LLM ώστε για κάθε ταινία να κάνει μια περίληψη. Έτσι οι 2 ερωτήσεις που θα έκανε το LLM για μια ταινία ως συνθετικό query η μία θα αφορούσε το αρχικό μεγάλο σενάριο που θα ήταν πιο περιγραφική και η 2η θα γινόταν πάνω στην περίληψη που είχε γίνει από το LLM. Όλα αυτά γίνονται vectors τα οποία στα metadata έχουν το τύπο του content με ξεχωριστή τιμή. Για κάθε ταινία μπορούν να υπάρχουν πολλά Chroma documents με κοινό movie_id, αλλά διαφορετικό doc_type στα metadata. Για παράδειγμα μια ταινία μπορεί να είναι doc τύπου summary, full_plot_semantic_query, summary_semantic_query, chunk_paragraph. Αυτό επιτρέπει στο σύστημα να αναζητά την ίδια ταινία από διαφορετικές σημασιολογικές αναπαραστάσεις. Μελλοντικά μπορούν να γίνουν διάφορα είτε πχ ένα LLM να φτιάχνει tags για ταινίες πχ "Psychological/emotional" είτε κάποιο clustering αν το σπάσουμε σε chunks. Οι ταινίες ήταν αρκετές αλλά επειδή ήταν πολλά και τα και τα queries και summaries που έπρεπε να παραξει το LLM επεξεργαστήκα διάφορα datasets μέσω της pandas και merge με τις αξιολογήσεις ώστε να φτιάξω ένα για data της εφαρμογής και κατέληξα σε κάποια οι ταινίες πρέπει να είναι γύρω στις 6.800 με 10.000 αναλογως ποιο dataset είχα χρησιμοποιήσει από αυτά που είχα παραξει σε csv. Υπάρχει στο package της εφαρμογής στο path app/data/movies_data.csv το dataset που έχω κάνει chunks τις παραγράφους αλλά δεν το προτείνω καθώς έχει 52.000 εγγραφές που περιέχουν τις περιλήψεις τα παραγόμενα query τα paragraph_chunks και αργεί πολύ να φτιαχτεί η βάση και να τα κάνει embeddings κι έτσι προτείνω το dataset που δεν έχει γίνει chunking κι έχει περιλήψεις και queries νομίζω είναι γύρω στις 22.000 εγγραφές app/data/movies_data.csv καθώς οι απαντήσεις είναι αρκετά καλές. Το άλλο είναι για μελλοντική επέκταση που αφορά το recommendation που θα εμφανίζεται στην ενότητα recommendation όταν θα κάνει favorite μια ταινία. Είναι διαφορετική ενότητα-καρτέλα οι προτάσεις που κάνει το AI μέσω RAG αναζήτησης σεναρίου ταινίας σε συζήτηση που ανοίγει sessions και διαφορετική η ενότητα recommendation που αφορά τα ενδιαφέροντα του χρήστη τα οποία συνδυάζονται και με ενδιαφέροντα άλλων χρηστών από dataset που βρήκα στο διαδίκτυο. Κάθε φορά που κάνει ο χρήστης μια ταινία favorite αυτή προτιμάται στην βάση στις αγαπημένες του με αξιολόγηση 5 για χαριν απλότητας της λογικής recommendation συνδυάζοντας με άλλους χρήστες τρέχει ένα background task μετά από κάθε κλικ στο favorite που αναλύει με item base collaborative filtering ώστε να του προτείνει στην καρτέλα recommendation. Δεν έχει αναπτυχθεί ιδιαίτερα ακόμα ο συνδυασμός με content. Εκεί θα χρησιμεύσουν τα chunks στο recommendation που θα μπορούσε να ήταν ένα clustering στα chunks των ταινιών που έχει ως αγαπημένες κι έτσι θα ομαδοποιήσουμε τα ξεχωριστά ενδιαφέροντα του. και μετά να του προτείνει τις ταινίες των οποίων τα chunks απλωνώνονται περισσότερο μεταξύ των cluster του ενδιαφέροντος του με κάποιο threshold. Δηλαδή μπορεί στα ενδιαφέροντα του να έχουν ξεχωρήσει κάποιο cluster με σκοτεινές ταινίες επιστημονικής φαντασίας, ενώ σε άλλο cluster να έχει ταινίες σχετικές που διαδραματίζονται σε ένα νησί. Έτσι αν μια ταινία είναι επιστημονικής φαντασίας που διαδραματίζεται σε ένα νησί, μπορεί ένα chunk πχ μια παράγραφος να μιλάει για το ότι είναι επιστημονικής φαντασίας ενώ σε άλλο chunk η παράγραφος

να περιγράψει την κατάσταση στο νησι που θα σημαίνει ότι 2 chunks από αυτήν την ταινία πέσανε πολύ κοντά σε κέντρα ενός cluster του. όσο πιο πολλά chunk μιας ταινίας πεφτούν κοντά στους διαφορετικούς cluster του τόσο πιο πάνω θα είναι στο recommendation. μπορεί να μπει ένα threshold για το ποσο μακριά από το κέντρο του cluster θα θεωρείται ότι έπεσε μέσα. Αυτό θα παίρνει μια βαθμολογία με έναν συντελεστή και θα συνδυάζεται με το item-base collaborative filtering που θα παίρνει κι αυτό ένα συντελεστή για το τελικό recommendation δηλαδή θα συνδυάζει την ομοιοτητα των αξιολογήσεων με τους άλλους χρήστες πχ με 0.5 και θα προσθέτει την βαθμολογία από το content των clusters με 0.5. Αλλά αυτό θα πρέπει να έχει κάνει favorite πολλές ταινίες για να έχει νοήματα ξεχωριστά ενδιαφέροντα του. δηλαδή να ενεργοποιηθεί μετά από αρκετά likes. Επίσης θα πρέπει να κάνει routing όλες τις υπολοίπες ταινίες για να βρει τις πιο semantic ενδιαφέρουσες πάνω στα clusters του κάθε χρήστη που αργεί λογικά αρκετά να εκτελεστεί αλλά θα γίνεται στο background. Πειραματικά θα ήθελα να δοκιμάσω καθώς υπάρχει ο dataset με αξιολογήσεις αν μπορεί να δουλέψει το PCA ίσως πιο γρήγορα αντί για clustering γιατί από όσο ξέρω καθαρίζει λίγο από θόρυβο και συμπυκνώνει τις διακυμανσεις δηλαδή τα διαφορετικά ενδιαφέροντα ενός χρήστη εξηγώντας πχ το 90% των δαιακυμανσεων. πχ αν ένας χρήστης έχει κάνει like favorite σε πολλές ταινίες μπορεί να φτιαχναμε ένα PCA fit_tranform πάνω σε αυτά τα chunks πορεί να προκυψουν πχ 8 components που να εξηγούν ενδεχόμενος 8 διαφορετικά ενδιαφέροντα του χρήστη εξηγώντας 90% της δαικυμανσης. Έτσι α προβαλούμε όλες τις υπολοίπες ταινίες στον χώρο του κάθε χρήστη.

Βασικό σενάριο χρήσης

Ένας χρήστης συνδέεται στην εφαρμογή και μπορεί να εκτελέσει τις παρακάτω βασικές ενέργειες:

- Να κάνει login με jwt token authentication ή signup.
- Να αναζητήσει ταινίες με βάση τον τίτλο από το mysql.
- Να δει πληροφορίες για μια ταινία, όπως τίτλο, έτος, σκηνοθέτη, cast, genres και plot από mysql.
- Να προσθέσει μια ταινία στα αγαπημένα του (mysql).
- Να ανοίξει AI chat session και να κάνει ερωτήσεις φυσικής γλώσσας.
- Να λάβει απάντηση από το AI σύστημα βασισμένη σε σχετικά retrieved movie documents (chroma).

Ο χρήστης συνδέεται στην εφαρμογή και μπορεί να δημιουργήσει ένα νέο search session. Μέσα στο session γράφει ένα ερώτημα σε φυσική γλώσσα, όπως για παράδειγμα: θέλω μια σκοτεινή ψυχολογική ταινία με ένταση και μυστήριο. Το σύστημα αναλύει το ερώτημα, το μετατρέπει σε καλύτερο semantic query, εξάγει πιθανά φίλτρα όπως genre, director ή year και στη συνέχεια αναζητά σχετικά documents στη Chroma vector database. Τα αποτελέσματα που επιστρέφονται από το retrieval δίνονται σε ένα LLM, το οποίο κρίνει ποιες ταινίες ταιριάζουν καλύτερα στο αίτημα του χρήστη. Η τελική απάντηση δεν είναι απλώς μια λίστα αποτελεσμάτων, αλλά μια αιτιολογημένη πρόταση ταινιών στα ελληνικά, βασισμένη αποκλειστικά στα retrieved movies. Ο χρήστης μπορεί επίσης να δει ταινίες, να τις προσθέσει στα αγαπημένα του και να λάβει recommendations με βάση προηγούμενες προτιμήσεις του. Ο admin μπορεί να προσθέτει ή να διαγράφει ταινίες, να κάνει εισαγωγή νέων ταινιών τόσο στη MySQL και το σενάριο γίνεται embedding στη Chroma, και να παρακολουθεί στατιστικά κόστους και token usage ανά ημέρα.

Λειτουργικές απαιτήσεις

Απαιτείται Docker desktop.

Η εφαρμογή υποστηρίζει εγγραφή και σύνδεση χρηστών, διαχωρισμό απλού χρήστη και admin χρήστη, δημιουργία και προβολή search sessions, συνομιλία με AI agent για αναζήτηση ταινιών και αποθήκευση ιστορικού συνομιλίας ανά session.

Υποστηρίζει semantic αναζήτηση σε vector database, φιλτράρισμα με βάση genres, excluded genres, director και year, αναζήτηση με τίτλο και προβολή ταινιών με pagination και προβολή plot ταινίας σε ξεχωριστό dialog.

Authentication και χρήστες

Η εφαρμογή υποστηρίζει authentication χρηστών. Ο χρήστης μπορεί να κάνει login και να έχει προσωπικά δεδομένα, όπως αγαπημένες ταινίες και chat sessions. Το backend χρησιμοποιεί JWT authentication ώστε κάθε protected request να συνδέεται με τον τρέχοντα authenticated user. Χρησιμοποιείται το current_user το οποίο ελέγχει για την αυθεντικοποίηση του χρήστη. Η αυθεντικοποίηση γίνεται με JWT authentication. Μετά το login το backend επιστρέφει ένα access_token, το οποίο αποθηκεύεται στο localStorage του web. Μέσα στο JWT payload έχουμε πληροφορίες όπως το sub, το exp και το is_superuser. Το frontend κάνει decode το token για να καταλάβει αν ο χρήστης είναι απλός user ή admin. Αυτό χρησιμοποιείται για να γίνει σωστό redirect μετά το login καθώς ο απλός χρήστης πηγαίνει στο user template, ενώ ο admin πηγαίνει στο admin panel. Όμως αυτός ο έλεγχος στο frontend χρησιμεύει κυρίως για σωστή εμπειρία χρήστη. Η πραγματική ασφάλεια πρέπει πάντα να γίνεται και στο backend, με ελέγχους όπως current_user.is_superuser.

Για τα HTTP requests χρησιμοποιήσαμε Angular HttpClient και HTTP interceptor. Ο interceptor προσθέτει αυτόματα σε κάθε request το header. Authorization κάνει με Bearer token. Έτσι δεν χρειάζεται κάθε service να γράφει χειροκίνητα το token. Όταν το frontend κάνει request σε endpoints όπως /movies, /users/favorites, /users/recommendations, /searches ή /admin/movies, το JWT στέλνεται αυτόματα.

Διαχείριση ταινιών

Η εφαρμογή αποθηκεύει ταινίες σε relational database. Κάθε ταινία περιλαμβάνει βασικά metadata:

- id
- title
- year
- director
- cast
- plot
- genres

Αγαπημένες ταινίες

Ο χρήστης μπορεί να προσθέτει ταινίες στα αγαπημένα του. Η σχέση χρήστη και ταινίας αποθηκεύεται σε link table, ώστε να υποστηρίζεται many-to-many σχέση μεταξύ users και movies.

User <-> UserMovieFavorite <-> Movie

AI chat sessions

Ο χρήστης μπορεί να δημιουργήσει chat sessions. Κάθε session κρατά ιστορικό ερωτήσεων και απαντήσεων, ώστε η εφαρμογή να υποστηρίζει συνεχή διάλογο και ερωτήσεις.

- Δημιουργία session
- Λίστα sessions
- Chat μέσα σε session
- Λήψη ιστορικού session
- Διαγραφή session

Semantic movie search

Το σύστημα δεν βασίζεται μόνο σε keyword search. Χρησιμοποιεί embeddings και Chroma vector database, ώστε να βρίσκει ταινίες με βάση σημασιολογική ομοιότητα. Έτσι μπορεί να εντοπίζει σχετικά movie plots ή summaries ακόμη και όταν οι ακριβείς λέξεις του query δεν υπάρχουν στα δεδομένα.

Τεχνολογίες που χρησιμοποιήθηκαν

Το backend υλοποιείται με FastAPI που μαθαμε καθώς είναι γρήγορο, υποστηρίζει async λειτουργίες και ταιριάζει καλά με APIs που καλούν LLMs, vector databases και database services. Για ORM χρησιμοποιείται SQLAlchemy, το οποίο συνδυάζει SQLAlchemy και Pydantic models, διευκολύνοντας τη δημιουργία database models και API schemas. χρησιμοποιω επισης Error Handler Middleware, Cost Tracker και Rate Limiter. Με custom errors όπως RateLimitError που μπορεί να το κληρονομήσει ένα RequestLimitError είτε ένα TokenLimitError

Η MySQL χρησιμοποιείται ως relational database για δομημένα δεδομένα και την επεξεργασία για postgres απλα επειδη την ειχα κατεβασμενη και ειμουν πιο εξοικειωμενος για πιο γρηγορη υλοποιηση, όπως users, movies, genres, favorites, ratings, search sessions και search history. Η Chroma χρησιμοποιείται ως vector database για αποθήκευση embeddings και semantic retrieval πάνω σε plots, summaries και generated semantic queries.

Για embeddings χρησιμοποιείται το μοντέλο text-embedding-3-small, το οποίο μετατρέπει τα κείμενα των ταινιών σε διανύσματα ώστε να μπορεί να γίνει semantic similarity search. Για LLM reasoning, summarization και generation χρησιμοποιούνται μοντέλα OpenAI, όπως gpt-4.1-mini και gpt-4o-mini, ανάλογα με το task.

Το LangChain χρησιμοποιείται για τη σύνδεση LLMs, embeddings, Chroma retrievers και Documents. Το LangGraph χρησιμοποιείται για την οργάνωση του RAG agent ως graph με διακριτά βήματα όπως query rewriting, filter extraction, retrieval και answer generation. Το frontend υλοποιείται με Angular η οποία είναι καλή για δομή και οργάνωση ώστε να υπάρχει καθαρή διαχείριση pages, components, services και authenticated user interaction καθώς και επειδή είχα περισσότερη εξοικείωση.

Χρησιμοποιώ το Docker για να δώσω δυνατότητα τρεξίματος της εφαρμογής αλλά σε πολύ dev λογική χρησιμοποιώντας αρκετά το local windows και όχι σε production αυτονομο deploy. Η εφαρμογή τρέχει μέσα σε Docker containers μαζί με υπηρεσίες όπως η MySQL, όμως ο source code δεν χρησιμοποιείται αποκλειστικά από το Docker image. Μέσω bind mount συνδέω τον τοπικό φάκελο του project στα Windows με το /app μέσα στο container. Κατά το build το Dockerfile μπορεί να έχει ήδη αντιγράψει τον κώδικα μέσα στο image με COPY . . . Όταν όμως ξεκινήσει το container και εφαρμοστεί το bind mount ./app, ο τοπικός Windows φάκελος προσαρτάται πάνω στο /app και ουσιαστικά καλύπτει το /app που υπήρχε

μέσα στο image. Δεν διαγράφεται το περιεχόμενο του image· απλώς όσο το mount είναι ενεργό, το container βλέπει στο /app τα αρχεία του Windows host.

Αυτό το χρησιμοποιώ σκόπιμα για development, επειδή μπορώ να αλλάζω τον κώδικα στα Windows και οι αλλαγές να γίνονται άμεσα ορατές μέσα στο container. Αυτό επιτρέπει να κάνω αλλαγές τοπικά και να τις βλέπει άμεσα το container ακόμα και χωρίς νέο build. Σε συνδυασμό με το unicorn η εφαρμογή θα μπορούσε να επανεκκινείται αυτόματα αν χρησιμοποιούσα πχ --reload όταν αλλάζει ο Python κώδικας. Έτσι χρησιμοποιώ το Docker για απομόνωση των dependencies, της MySQL και του runtime περιβάλλοντος, αλλά συνεχίζω να αναπτύσσω τον κώδικα απευθείας από τα Windows. Εκτελείται με το uv το οποίο κατεβαίνει μέσω του docker συνεπώς δεν χρειάζεται ενώ παρέχω το pyproject.toml και το συγγεκριμένο uv.lock που έχει φτιαχτεί.

Error Handler Middleware

Το Error Handler Middleware χρησιμοποιείται για centralized error handling σε όλη την εφαρμογή. Αντί κάθε endpoint να επιστρέφει διαφορετική μορφή σφάλματος, το backend ορίζει κοινό format απάντησης μέσω του ErrorResponse schema. Αυτά τα components βοηθούν ώστε το API να επιστρέφει πιο καθαρά error responses, να καταγράφει τη χρήση tokens και το κόστος των AI calls, και να μπορεί να περιορίζει τη συχνότητα των requests.

Το βασικό error response έχει τη μορφή:

```
class ErrorResponse(BaseModel): error_type: str message: str details: Optional[dict] = None
```

Έτσι τα errors επιστρέφονται με σταθερή δομή, κάτι που βοηθά τόσο το frontend όσο και το debugging.

Η function setup_error_handlers(app) συνδέεται στο FastAPI app μέσα στο main.py:

```
setup_error_handlers(app)
```

και καταχωρεί custom handlers για διαφορετικές κατηγορίες σφαλμάτων.

RateLimitError Handler

Ο handler για RateLimitError χρησιμοποιείται όταν κάποιο request ξεπερνά τα επιτρεπτά όρια χρήσης. Επιστρέφει status code 429 και περιλαμβάνει πληροφορία retry_after, ώστε ο client να ξέρει μετά από πόσο χρόνο μπορεί να ξαναδοκιμάσει.

Παράδειγμα error type:

- REQUEST_LIMIT_429_AI_CALL
- RATE_LIMIT_429_AI_CALL
- TOKEN_LIMIT_429_AI_CALL

Αυτό το format κάνει το error πιο περιγραφικό, επειδή δείχνει:

το είδος του limit το HTTP status code το context στο οποίο συνέβη το error

RequestValidationError Handler

Ο handler για RequestValidationError διαχειρίζεται validation errors από το FastAPI Pydantic validation. Αν το request body ή τα query parameters δεν έχουν σωστή μορφή, επιστρέφεται structured response με τα προβληματικά πεδία και τα αντίστοιχα μηνύματα.

Αυτό κάνει τα validation errors πιο εύκολα αναγνώσιμα από το frontend.

HTTPException Handler

Ο handler για HTTPException καλύπτει τα συνηθισμένα HTTP errors που σηκώνονται μέσα στα endpoints, όπως:

401 Unauthorized 403 Forbidden 404 Not Found 409 Conflict

Αντί να επιστρέφεται τυχαία μορφή response το error γίνεται custom μέσω του ErrorResponse.

Συνολικά, το error handling layer κάνει το API πιο προβλέψιμο, πιο ασφαλές και πιο εύκολο στο debugging.

Το CostTracker κρατά in-memory συνολικά στατιστικά για όσο τρέχει το application process. Από την άλλη, το SearchHistory αποθηκεύει usage και estimated cost μόνιμα στη βάση δεδομένων.

Άρα υπάρχουν δύο επίπεδα παρακολούθησης:

CostTracker: in-memory runtime monitoring

SearchHistory: persistent per-request history στη βάση

Αυτό σημαίνει ότι το CostTracker είναι χρήσιμο για γρήγορα runtime analytics, αλλά μηδενίζεται αν γίνει restart το application. Για μόνιμα analytics χρησιμοποιείται ο πίνακας search_history

RateLimiter

Το RateLimiter είναι ένα απλό in-memory rate limiting component που μπορεί να χρησιμοποιηθεί για να περιορίζει πόσα requests μπορεί να κάνει ένας client μέσα σε συγκεκριμένο χρονικό παράθυρο.

Αν κι ακόμα δεν έχω φτάξει tokens per minute παρα μοναχα requests_per_minute = 10 Η βασική ρύθμιση γίνεται μέσω του RateLimitConfig:

```
@dataclass class RateLimitConfig: requests_per_minute: int = 10 tokens_per_minute: int = 100000
```

Η βασική λογική του limiter βασίζεται σε sliding window ενός λεπτού. Για κάθε client κρατά timestamps των πρόσφατων requests και ελέγχει αν ο αριθμός τους ξεπερνά το επιτρεπτό όριο.

Αρχιτεκτονική εφαρμογής και ροή δεδομένων

Η εφαρμογή ακολουθεί πολυεπίπεδη αρχιτεκτονική. Το frontend επικοινωνεί με το backend μέσω REST API endpoints. Το backend είναι υπεύθυνο για authentication, business logic, database operations, retrieval και επικοινωνία με τα LLMs.

Η MySQL κρατά τα βασικά relational δεδομένα της εφαρμογής. Για παράδειγμα, οι πίνακες movie, genre, user, search_session, search_history, movie_rating και user_movie_favorite αποθηκεύουν πληροφορίες για ταινίες, χρήστες, συνομιλίες και προτιμήσεις. Η Chroma κρατά τα semantic documents που χρησιμοποιούνται στο RAG retrieval.

Αφου ο χρήστης κάνει login στην αρχικη σελίδα μεαφερεται σε ενα template που φαινονται σε λιστα με pagination ολες οι ταινιες και να δει πληροφοριες οπως Title,Genres,Director,Year,Plot καθώς και να κανει μια ταινια favorite με κουμπι. Επισης υπαρχει μπαρα αναζητησης που μπορεί να αναζητηση με βαση τον τιτλο. Αριστερα εχει ενα μενου πλοηγησης που μπορεί να αλλαξει σελιδα απο την αρχικη που περιλαμβανει

τις ενοητες Ask AI, Recommendations, Favorites. Η ενοτητα Ask AI είναι αυτή που του επιτρέπει να συνομιλήσει με το LLM και να αναζητήσει ταινίες με βάση το σενάριο που ψάχνει. Αφού πλοηγηθεί ενοτητα αυτή, για να μπορέσει να ανοίξει συζήτηση πρέπει να πατήσει ένα κουμπί το οποίο δημιουργεί το session στέλνοντας το κατάλληλο request στο backend. Με αυτόν τον τρόπο του ανοίγει ένα πλαίσιο να μπορεί να πληκτρολογήσει και να πατήσει send και στέλνεται η ερώτηση με το id του Session που ανοίξε. Έτσι ξεκινάει ο γράφος και μόλις ολοκληρωθεί επιστρέφει την απάντηση. Η ενοτητα recommendations είναι έχει τις προτάσεις των ταινιών προς τον χρήστη με βάση αυτές που έχει κάνει favorite και η ενοτητα favorites βρίσκονται οι ταινίες που έχει πατήσει favorite.

Για κάθε ταινία μπορούν να υπάρχουν πολλά Chroma documents με κοινό movie_id, αλλά διαφορετικό doc_type. Για παράδειγμα, μια ταινία μπορεί να έχει document τύπου full_plot, summary, full_plot_semantic_query ή summary_semantic_query. Αυτό επιτρέπει στο σύστημα να αναζητά την ίδια ταινία από διαφορετικές σημασιολογικές αναπαραστάσεις.

Ειδική περίπτωση ροής για προσθήκη αγαπημένης ταινίας

οταν ερθει ενα request για like favorite σε μια ταινια τοτε αυτη αποθηκευεται στο mysql στις αγαπημενες ταινιες του χρηστη αλλα ταυτόχρονα της δινεται και η αξιολογηση με 5 απο τον χρηστη που είναι μεγιστη τιμη ωστε να λειτουργησει το recommendation system. Μολις ερθει το request ξεκινάει να εκτελειται ξεχωριστα ενα background task που έχει

Περιγραφή LangGraph RAG

- Εδώ θα περιγράψω αναλυτικά το state, το vectorstore, το retrieve strategy και το κύκλο του graph

To State Graph

```
class MovieRAGState(TypedDict): user_query: str chat_history: list[dict[str, str]]
```

```
semantic_query: str
rewrite_reasoning: str

genres: list[str]
excluded_genres: list[str]
director: Optional[str]
year: Optional[int]
needs_filter: bool
filter_reasoning: str

retrieved_movies: list[dict]
final_answer: str

needs_retrieval: bool
routing_reasoning: str
```

```
usage: dict[str, int] # {input_tokens:int, output_tokens:int, total_tokens:int}
cost: float
```

- **user_query**: Είναι το τελευταίο μήνυμα που έστειλε ο χρήστης. Αυτό είναι το κύριο input του γράφου.
- **chat_history**: Περιέχει προηγούμενα μηνύματα του session τα παίρνει απο το memory του SearchSession που τα κρατάει ως string λιστα `str = Field(default="[]")` στο model `mySQL`. Χρησιμοποιείται ώστε ο agent να μπορεί να καταλάβει ερωτήσεις όπως και κάτι πιο σκοτεινό. κρατάει μονο τα τελευταία `messages max_messages = 10`
- **semantic_query**: Είναι το `rewritten query` που δημιουργείται από τον `rewrite` κόμβο. Είναι πιο κατάλληλο για `semantic retrieval` από το αρχικό μήνυμα του χρήστη.
- **rewrite_reasoning**: Περιέχει την αιτιολόγηση του `rewrite LLM`, δηλαδή γιατί μετέτρεψε το query με συγκεκριμένο τρόπο.
- **genres, excluded_genres, director, year**: Δηλώνει αν το query περιέχει χρήσιμα `metadata filters`.
- **filter_reasoning**: Περιέχει την αιτιολόγηση του `filter extraction`.
- **retrieved_movies**: Περιέχει τα `movie results` που επιστρέφει το `retrieval`.
- **final_answer**: Είναι η τελική απάντηση που θα επιστραφεί στον χρήστη.
- **needs_retrieval**: Αποφασίζει αν το μήνυμα χρειάζεται νέα αναζήτηση ταινιών ή όχι.
- **routing_reasoning**: Περιέχει την αιτιολόγηση του `retrieval router`.
- **usage και cost**: Το state παρακολουθεί και ενημερώνει `tokens` και εκτιμώμενο κόστος. Η `helper _update_usage_state_format` παίρνει `usage` από κάθε `LLM response`, προσθέτει `tokens` στο προηγούμενο `usage` και υπολογίζει επιπλέον κόστος.

Vectorstore

Το project χρησιμοποιεί `Chroma vectorstore` για `semantic search` πάνω σε `movie documents`. Το `vectorstore` αποθηκεύεται τοπικά στον φάκελο:

```
app / chroma_db
```

Η `collection` ονομάζεται `movies` Η `Chroma` χρησιμοποιείται μαζί με `OpenAIEmbeddings`, συγκεκριμένα με το `model text-embedding-3-small` Κάθε `document` μετατρέπεται σε `embedding vector` και αποθηκεύεται στη `Chroma` μαζί με `metadata` που επιτρέπουν `filtering`, εμφάνιση πληροφοριών και σύνδεση του `vector result` με την αντίστοιχη ταινία στη βάση.

Κάθε εγγραφή στο `vectorstore` περιέχει τρία βασικά πράγματα:

1. `page_content`
2. `embedding`

3. metadata

page_content

Το **page_content** είναι το πραγματικό κείμενο που γίνεται embedded. Αυτό μπορεί να είναι plot, summary, paragraph, synthetic query ή άλλο κείμενο σχετικό με την ταινία, ανάλογα με το **doc_type**. Αν το **page_content** είναι κενό ή nan, το document παραλείπεται και δεν μπαίνει στη Chroma.

Embeddings

Για κάθε **page_content**, δημιουργείται embedding με `OpenAIEmbeddings(model="text-embedding-3-small")` Έτσι, όταν ο χρήστης κάνει ένα query όπως "Θέλω μια σκοτεινή sci-fi ταινία με φιλοσοφικό ύφος το query μετατρέπεται και αυτό σε embedding και συγκρίνεται με τα αποθηκευμένα embeddings των movie documents.

Chroma IDs

Κάθε document αποθηκεύεται με μοναδικό Chroma id:

```
chroma_id = f"{idx}_{doc_type}"
```

Δηλαδή το id αποτελείται από: CSV row index + doc_type πχ:

```
15_summary  
16_full_plot  
17_synthetic_query
```

Αυτά τα ids χρησιμοποιούνται από τη Chroma για να ξεχωρίζει τα documents. Ο κώδικας ελέγχει επίσης αν υπάρχουν duplicate ids:

```
if len(set(chroma_ids)) != len(chroma_ids):  
    print("WARNING: You have duplicate ids!")
```

Metadata

Κάθε document αποθηκεύεται μαζί με metadata. Τα metadata βοηθούν στο filtering, στο debugging και στη σύνδεση των retrieved documents με τις ταινίες.

Η βασική μορφή metadata είναι:

```
metadata = {  
    "movie_id": movie_id,  
    "title": title,  
    "genres": genres_raw,  
    "year": year,  
    "director": director,  
    "source_movie": source_movie,  
    "doc_type": doc_type,  
}
```

movie_id

Το **movie_id** συνδέει το document της Chroma με την αντίστοιχη ταινία στη relational database. Αυτό είναι πολύ σημαντικό, γιατί γίνονται ταυτόχρονα 2 retrieval που επιστρέφουν documents, αλλά η εφαρμογή συχνά χρειάζεται να ξέρει σε ποια ταινία ανήκει κάθε document.

`title

Το **title** αποθηκεύει τον τίτλο της ταινίας.

genres

Το **genres** αποθηκεύεται ως raw string, συνήθως με separator `|` κυρίως για display αν χρειαζοταν όχι για filtering. πχ:

```
"genres": "Action|Sci-Fi|Thriller"
```

Για filtering χρησιμοποιούνται ξεχωριστά boolean genre flags.

Genre metadata flags

Για κάθε genre δημιουργείται ξεχωριστό metadata field.

Ο κώδικας παίρνει τα genres:

```
genres = [g.strip() for g in genres_raw.split("|") if g.strip()]
```

και για κάθε genre δημιουργεί field:

```
metadata[f"genre_{normalize_key(genre)}"] = True
```

πχ μια ταινία έχει genres:

```
Comedy | Drama | Romance
```

τότε το metadata θα περιέχει:

```
{  
  "genre_comedy": True,  
  "genre_drama": True,  
  "genre_romance": True  
}
```

Αυτό χρησιμεύει στο metadata filtering της Chroma. Δηλαδή, αν ο χρήστης ζητήσει comedy movies, το retrieval μπορεί να φιλτράρει με βάση:

```
{"genre_comedy": True}
```

Τα genres που δεν ανήκουν στην ταινία συνήθως δεν αποθηκεύονται δεν υπάρχουν καθόλου ως keys στο metadata σαν να είναι NaN.

Η function `normalize_key()` χρησιμοποιείται για να μετατρέπει τα genre names σε ασφαλή metadata keys.

Παράδειγμα:

```
"Science Fiction" -> "science_fiction"  
"Film-Noir" -> "film_noir"  
"Action & Adventure" -> "action_adventure"
```

Έτσι δημιουργούνται σταθερά metadata fields όπως:

```
genre_science_fiction  
genre_film_noir  
genre_action_adventure
```

doc_type

Το `doc_type` δηλώνει τι είδους document είναι το συγκεκριμένο Chroma entry.

Στον κώδικα διαβάζεται από τη στήλη:

```
row.get("doc_type", "no_type")
```

Αν δεν υπάρχει `doc_type`, τότε παίρνει default τιμή `no_type`.

Το `doc_type` είναι σημαντικό γιατί η Chroma δεν αποθηκεύει μόνο ένα είδος κειμένου για κάθε ταινία. Μπορεί να αποθηκεύει διαφορετικές αναπαραστάσεις της ίδιας ταινίας, ώστε το retrieval να είναι πιο ευέλικτο. Αυτό γίνεται γιατί διαφορετικά queries ταιριάζουν καλύτερα με διαφορετικές μορφές κειμένου. Ένα γενικό query μπορεί να ταιριάζει καλύτερα με summary η μια παραγραφο αν κανουμε chunk πιανοντας περισσότερο το γενικο νοημα. Ένα query γραμμένο σαν φυσική αναζήτηση χρήστη μπορεί να ταιριάζει πολύ καλά με ένα generated query.

Ενδεικτικά στο project μπορούν να υπάρχουν doc types όπως:

```
summary
full_plot_semantic_query, summary_semantic_query
paragraph
```

Η ακριβής λίστα εξαρτάται από τις τιμές της στήλης `doc_type` στο CSV που έχω παραξει. Το `summary` περιέχει σύντομη περίληψη της ταινίας που έχει γραφτεί από ένα LLM. Χρησιμεύει όταν θέλουμε πιο γενικό semantic matching. Είναι πιο συμπυκνωμένο από το πλήρες plot και συχνά βοηθάει στο να βρεθεί γρήγορα η θεματική της ταινίας. Για παραδειγμα "ταινίες με θέμα τεχνητή νοημοσύνη και ταυτότητα" το summary μπορεί να ταιριάζει καλά επειδή περιέχει τη βασική ιδέα της ταινίας. Έτσι μπορεί να γίνει retrieve με φίλτρο `doc_type="summary"`

Το paragraph ή chunk-like document περιέχει κομμάτι από μεγαλύτερο plot. Παροτι προς το παρων προτεινω την υλοποιηση χωρις chunk paragraphs χαριν ταχυτητας. Για παραδειγμα σύστημα μπορεί να βρει ένα συγκεκριμένο σημείο της πλοκής με βάση το query του χρηστη, αντί να συγκρίνει ολόκληρο το plot σαν ένα μεγάλο κείμενο. Χρησιμεύει όταν το πλήρες plot είναι μεγάλο και θέλουμε πιο λεπτομερή retrieval σε μικρότερα τμήματα.

Το `semantic_query` είναι generated by llm query document. Ένα LLM έχει προσομοιασει μια ενδεχομενη ερωτηση χρηστη προς αυτην την ταινια. Δηλαδή αντί να αποθηκεύεται μόνο περιγραφή της ταινίας, αποθηκεύεται και ένα πιθανό ερώτημα χρήστη που θα μπορούσε να αντιστοιχεί σε αυτή την ταινία. Υπάρχουν 2 ειδων semantic_query ένα που έχει παραχθεί πάνω στο full plot -> `full_plot_semantic_query` κι ένα που έχει παραχθεί πάνω στην περιληψη -> `summary_semantic_query` Αυτό βοηθάει όταν ο χρήστης ρωτά με φυσική γλώσσα. Το query του χρήστη μπορεί να μοιάζει περισσότερο με synthetic query παρά με το πραγματικό plot. Παράδειγμα semantic queries για την ίδια ταινία The prophecy:

```
"What movie features a detective who confronts a war between angels and struggles with his faith after witnessing disturbing visions?" - full_plot_semantic_query
```

```
"What movie features a detective who loses his faith while investigating a celestial war between angels over a deceased veteran's soul?"
```

Μία ταινία μπορεί να έχει πολλαπλά entries στη Chroma.

```
movie_id = 500
500_summary
500_full_plot_semantic_query
500_summary_semantic_query
500_paragraph_1
500_paragraph_2
```

year

Το **year** αποθηκεύει το έτος της ταινίας, αν υπάρχει.

```
"year": 1999
```

Κατα την δημιουργία του vectorstore απο το csv Αν το year είναι κενό ή NaN, μετατρέπεται σε None και μετά αφαιρείται από τα metadata επειδή στο τέλος γίνεται

```
metadata = {k: v for k, v in metadata.items() if v is not None}
```

None δεν αποθηκεύονται στη Chroma.

director

Το **director** αποθηκεύει τον σκηνοθέτη της ταινίας. Αν δεν υπάρχει τιμή, η helper **safe_str()** το μετατρέπει σε κενό string:

```
"director": ""
```

Μπορεί να χρησιμοποιηθεί είτε για display είτε για filtering, αν το retrieval pipeline υποστηρίζει director filters.

source_movie

Το **source_movie** αποθηκεύει επιπλέον πληροφορία προέλευσης για το document.

Μπορεί να χρησιμοποιηθεί για να δηλώσει από ποια ταινία ή ποιο αρχικό movie record προήλθε ένα generated document. Αν δεν υπάρχει τιμή, αποθηκεύεται ως κενό string.

Πώς χρησιμοποιείται στο RAG

Το vectorstore χρησιμοποιείται στο retrieval βήμα του RAG pipeline.

Η ροή είναι:

```
User query
  |
  v
Query rewriting / semantic query
  |
  v
Embedding του query
  |
  v
Search στη Chroma collection "movies"
  |
  v
Retrieved movie documents
  |
  v
LLM final answer
```

Η Chroma επιστρέφει τα πιο σημασιολογικά κοντινά documents. Μετά το σύστημα χρησιμοποιεί τα metadata, όπως `movie_id`, `title`, `year`, `genres`, `director` και `doc_type`, για να οργανώσει τα αποτελέσματα και να δημιουργήσει τελική απάντηση.

Πώς χρησιμοποιείται στο filtering

Τα metadata επιτρέπουν στο retrieval να συνδυάζει semantic search με structured filters.

Για παράδειγμα, αν ο χρήστης ζητήσει:

```
Θέλω κωμωδία από το 1999
```

το σύστημα μπορεί να χρησιμοποιήσει:

```
semantic_query = "comedy movie"
filters = {
    "genre_comedy": True,
    "year": 1999
}
```

Έτσι δεν ψάχνει απλά για παρόμοια embeddings, αλλά περιορίζει τα αποτελέσματα σε documents που ανήκουν σε comedy movies και έχουν συγκεκριμένο έτος.

Batching

Τα documents προστίθενται στη Chroma σε batches των 500: Για κάθε batch, το script καλεί:

```
vectorstore.add_documents(  
    documents=batch_docs,  
    ids=batch_ids,  
)
```

Summary

Το Chroma vectorstore της εφαρμογής αποθηκεύει movie-related documents μαζί με embeddings και metadata. Τα embeddings χρησιμοποιούνται για semantic search, ενώ τα metadata χρησιμοποιούνται για σύνδεση με τη βάση, εμφάνιση πληροφοριών και filtering.

Κάθε document περιέχει:

```
page_content -> το κείμενο που γίνεται embedded  
embedding -> το vector representation του κειμένου  
metadata -> πληροφορίες για movie_id, title, year, director, genres και doc_type
```

Τα **doc_type** επιτρέπουν στην εφαρμογή να αποθηκεύει διαφορετικές μορφές πληροφορίας για την ίδια ταινία, όπως summary, full plot, paragraph ή synthetic query. Τα genre flags όπως **genre_comedy=True** χρησιμοποιούνται για αποτελεσματικό metadata filtering, ενώ τα raw genres κρατούνται κυρίως για display.

Με αυτόν τον τρόπο, το vectorstore λειτουργεί ως η semantic retrieval βάση του RAG συστήματος και επιτρέπει στον agent να βρίσκει ταινίες με βάση φυσικές ερωτήσεις του χρήστη, όχι μόνο με keyword search.

Πως γίνεται το retrieve.

Η λογική του retrieve που αναλυεται παρακατω υπαρχει σαν κωδικας στο component app\rag\retrieve.py Αυτος καλειται και χρησιμοποιειται απο τον γραφο μεσα στο app\rag\graph.py οταν χρειάζεται. Το graph

Για κάθε ταινία μπορούν να υπάρχουν πολλά Chroma documents με κοινό movie_id, αλλά διαφορετικό doc_type στα metadata. Για παράδειγμα μια ταινία μπορεί να είναι doc τύπου summary, full_plot_semantic_query, summary_semantic_query, chunk_paragraph. Αυτό επιτρέπει στο σύστημα να αναζητά την ίδια ταινία από διαφορετικές σημασιολογικές αναπαραστάσεις. Δηλαδή αν προκειται για περιληψη -> doc_type="summary", αν προκειται για το llm-generated query -> doc_type = "full_plot_semantic_query"> ή "summary_semantic_query", αν προκειται για παραγραφο -> doc_type = "chunk_paragraph". Η ιδέα είναι να γινονται concurrently πολλα retrieve tasks αναζητώντας με filtering με βάση το doc_type και επιστρεφει ασυγχρονα concurrently τα αποτελεσματα. Για χαριν απλοτητας προς το

παρων κάνει `asyncio.gather 2 retrieval tasks`. Το ένα κάνει `retrieve` με `filter` την περίληψη και το άλλο `retrieve` επιστρέφει τα παραγόμενα από το `llm queries`. Αν το `id` μιας ταινίας εμφανίζεται πολλές φορές στο `retrieval` δηλαδή σε όλα τα `retrieve tasks` σημαίνει είναι και πιο σχετικό ώστε να προταθεί στον χρήστη. Αυτό γίνεται γιατί διαφορετικά `queries` ταιριάζουν καλύτερα με διαφορετικές μορφές κειμένου.

- Ένα γενικό `query` μπορεί να ταιριάζει καλύτερα με `summary` ή μια παραγραφο πιανοντας περισσότερο το γενικό νοημα.
- Ένα `query` γραμμένο σαν φυσική αναζήτηση χρήστη μπορεί να ταιριάζει πολύ καλά με ένα `generated query`. Εδώ μελλοντικά μπορεί να γίνει ένα καλύτερο `reranking evaluation` αλλά για χαριν απλοτητας προς το παρων τα αξιολογει ένα `LLM` που παιρνει και τα `metadata` στο `content`. Επίσης να σημειωσω οτι στο `retrieve task` που επιστρέφει τα παραγομενα `queries` δεν εβαλα καποιο φιλτρο γιατι ουτως η αλλος επιστρέφει συνηθως `queries` ως τα πιο σχετικα απο οτι παρατηρησα εμπειρικά και το αφησα μπας και επιστρέψει καμια παραγραφο και χαριν απλοτητας μην μπλεκω με πολλα `tasks`. Ετσι το φιλτρο μπηκε μονο στο `retrieval` για `summary` απλα αυτη ηταν η γενικη ιδεα.

Ο γράφος αναλυτικά.

Ο γράφος χρησιμοποιεί `StateGraph` από `LangGraph`, `LangChain message objects`, `OpenAI chat model` και `structured outputs` για ενδιάμεσα βήματα όπως `routing`, `rewrite` και `filter extraction`. Τα βασικά `imports` περιλαμβάνουν `StateGraph`, `START`, `END`, `LangChain messages`, `ChatOpenAI` `Chroma` καθώς και τα `structured output schemas` `MovieFiltersOutput`, `RetrievalDecisionOutput`, `SemanticRewriteOutput`. Χρησιμοποιει `gpt-4.1-mini temperature=0`, άρα η συμπεριφορά του είναι πιο σταθερή και `deterministic` για `structured` εργασίες όπως `routing`, `query rewriting` και `filter extraction`, ενώ σε κάθε βημα του γραφου που θα χρειαστει το `llm` ενημερωνει στο `state` το `usage` με τα `input output`, `total tokens` καθώς και το `cost` ώστε να τα δώσει στον `cost tracker` που έχω τιαξει για να εκτιμησει το τελικο κοστος του `request`.

Ο γράφος υλοποιεί ένα `RAG assistant` με `LangGraph`. Αρχικά δέχεται μήνυμα χρήστη μέσα σε ένα `chat-search session` αποφασίζει αν χρειάζεται νέα αναζήτηση ταινιών ή αν μπορεί να απαντήσει απευθείας με βάση το υπάρχον ιστορικό ή αν προκειται για απλη κουβεντα με το `AI` του τυπου "καλησπερα βοηθε μου".

Αν χρειάζεται αναζήτηση:

- κάνει `rewrite` του `query` γραφωντας το στα αγγλικά απαλοιφοντας τον θόρυβο όπως πχ "βρες μου αστυνομικες ταινία επιστημονικής φαντασίας με φιλοσοφικές προεκτάσεις" αφαιρει τις λέξεις "βρες μου" και το κάνει "Sci-fi detective movie with philosophical themes",
- Αναγνωρίζει κι εξάγει φίλτρα: πχ "Scient fiction detective movie with philosophical themes" Αναγνωρίζει το `Scient fiction` και το κάνει `Sci-Fi` για `filtering retrieval`. Τα είδη που μπορεί να εξάγει είναι συγκεκριμένα και του έχουν δοθεί στο `system prompt` πχ αν στο `system prompt` δεν του ειχα δώσει ως είδος το `Romance` και επεφτε μια ταινια `romance` δεν θα εκανε εξαγωγή φίλτρου. Επίσης είναι συγκεκριμένης μορφης:
- "kids movie" -> "Children"
- "animated movie" -> "Animation" Επίσης κάνει και φίλτρο `director`, πιο ελευθερο χωρις να του δωσω λιστα με συγκεκριμενους `directors`. Επίσης μπορεί ο χρήστης να αναζητησει μια ταινία με περιγραφη που όμως να γραψει να μην είναι `sci-fi`, τότε αναγνωρίζει και φτιαχνει φίλτρο ώστε να εξαιρει το `sci-fi` κι οχι να κάνει αναζητηση με `sci-fi`. Το `year` δεν το έχω φτιαξει όπως πρεπει ακομα, υπαρχει μεν στα φίλτρα αλλά αμα του πει θελω ταινιες μετα το 2005 δεν έχει φίλτρο αλλά θα το εξετασει το τελικο `LLM` που δινει την απαντηση ομως το σωστο ηταν να κάνει εκεί αναζητηση με φίλτρο `>=2005` αλλά

τώρα το έχω μόνο σαν εξαγωγή φίλτρου αναζήτησης με βάση την χρονολογία πχ "θέλω ταινίες του 2005". ,

- κάνει semantic retrieval από το vectorstore με βάση τα φίλτρα και το query που φτιαχτηκε. Γίνονται ταυτόχρονα 2 concurrently retrieve tasks με φίλτρο τι τύπου doc να ψάξει όπως πχ semantic query που έχει παραχθεί από LLM για κάθε σεναριο ταινίας προσομοιάζοντας την ερώτηση του χρήστη που θα έπρεπε να κάνει ώστε να του επιστρέψει αυτήν την ταινία, είτε την περίληψη της ταινίας που έχει γραφτεί εξίσου από ένα LLM σαν γενικό νόημα είτε αν το επεκτείνουμε με chunks. Στην προκειμένη περίπτωση το ένα retrieve task γίνεται με φίλτρο το doc να είναι περίληψη που έχει παραχθεί από LLM, ενώ στο άλλο retrieve task χωρίς φίλτρο επιστρέφει συνήθως σχεδόν πάντα πρώτα πρώτα τις προσομοιωμένες από LLM ερωτήσεις του χρήστη είτε θα μπορούσε να επιστρέψει και chunks αν το φτιάξω λίγο καλύτερα
- Τέλος ένα LLM λαμβάνει ως content όλα τα retrieved docs καθώς και τα φίλτρα αν αποτελούν πχ παραγόμενο query κτλ είτε περίληψη πλοκής κι αξιολογεί ποια είναι τα πιο σχετικά με την ερώτηση του χρήστη ώστε να του απαντήσει και αποθηκεύεται και στο model mysql στο searchHistory και στο searchSession στο memory η συζήτηση ενώ με την ολοκλήρωση του κάθε γραφού που σημαίνει κάθε request, ενημερώνεται και το usage των tokens και το cost τόσο στον cost tracker όσο και στο sql model στο πίνακα search_history που κρατάει κάθε ερωτοαπάντηση δηλαδή κάθε ερώτηση του χρήστη μέχρι την απάντηση του llm είναι μια εγγραφή που έχει στο history το usage και cost

Ο γράφος χρησιμοποιεί StateGraph από LangGraph, LangChain message objects, OpenAI chat model και structured outputs για ενδιάμεσα βήματα όπως routing, rewrite και filter extraction. Τα βασικά imports περιλαμβάνουν StateGraph, START, END, LangChain messages, ChatOpenAI, Chroma, καθώς και τα structured output schemas MovieFiltersOutput, RetrievalDecisionOutput, SemanticRewriteOutput.

```

User στέλνει ερώτηση
|
FastAPI λαμβάνει query και session_id
|
Το query περνάει από LangGraph workflow
|
Ένα llm αποφασίζει αν χρειάζεται retrieve ή όχι. αν δεν χρειάζεται δίνει απάντηση
και ολοκληρώνει τον γράφο
|
Γίνεται rewrite το query για απαλοιφή θορύβου
|
Γίνεται extraction πιθανών φίλτρων
|
2 concurrent filtering retrieve tasks στη Chroma vector DB
|
Ανακτώνται σχετικά movie documents
|
Το LLM λαμβάνει query + retrieved context αξιολογεί
|
Παράγεται τελική απάντηση
|
Η απάντηση αποθηκεύεται στο history ενημερώνει το usage

```

Επιστρέφεται στο frontend

FastAPI endpoints και βασικά services

Public area: / /login /register

User area: /user/movies /user/ask-ai /user/recommendations /user/favorites

Admin area: /admin /admin/movies /admin/movies/new /admin/analytics

```
router = APIRouter(prefix="/searches", tags=["searches"])
```

- GET /searches/ Επιστρέφει τις search/conversation sessions. Αν ο χρήστης είναι superuser, βλέπει όλες τις sessions της εφαρμογής· διαφορετικά βλέπει μόνο όσες έχουν owner_id == current_user.id. Υποστηρίζει pagination μέσω skip και limit και επιστρέφει SearchSessionsPublic, δηλαδή τις sessions μαζί με το συνολικό count.
- GET /searches/{id} Επιστρέφει μία συγκεκριμένη search session μαζί με ολόκληρο το history της. Αρχικά βρίσκει τη SearchSession από το UUID. Αν δεν υπάρχει επιστρέφει 404, ενώ αν δεν ανήκει στον τρέχοντα χρήστη επιστρέφει 403. Μετά διαβάζει όλα τα SearchHistory της συγκεκριμένης session ταξινομημένα χρονικά (created_at.asc()) και τα επιστρέφει ως SearchSessionWithHistoryPublic.
- POST /searches/ — create_search() Δημιουργεί μία καινούργια search conversation session. Παίρνει τα δεδομένα από SearchSessionCreate και προσθέτει αυτόματα ως owner_id τον authenticated χρήστη. Κάνει commit, refresh και επιστρέφει τη νέα session ως SearchSessionPublic.
- POST /searches/{id}/chat — chat_with_agent_on_search() Συνεχίζει τη συζήτηση μιας υπάρχουσας session με τον AI agent. Πρώτα εφαρμόζει rate limiting βάσει του IP (10 requests/minute), ελέγχει ότι υπάρχει η session και ότι ανήκει στον χρήστη, και απορρίπτει κενό message. Στη συνέχεια καλεί await run_agent_on_session() περνώντας τη συγκεκριμένη SearchSession, τον χρήστη το νέο μήνυμα και τον cost_tracker. Άρα το graph εκτελείται στο context της συγκεκριμένης συζήτησης και μπορεί να χρησιμοποιήσει το προηγούμενο history memory που σχετίζεται με αυτή. Επιστρέφει

AgentChatResponse(session_id=..., reply=...)

- DELETE /searches/{id} — delete_search() Διαγράφει μία συγκεκριμένη search session. Ελέγχει πρώτα ότι υπάρχει (404) και ότι ανήκει στον τρέχοντα χρήστη (403). Μετά κάνει session.delete(search_session) session.commit()

και επιστρέφει μήνυμα επιτυχίας.

```
router = APIRouter(prefix="/users", tags=["users"])
```

Authentication / User endpoints

- POST /login

Χρησιμοποιείται για σύνδεση χρήστη με OAuth2 password . Παίρνει email και password μέσω form data, βρίσκει τον χρήστη από το email και ελέγχει το password. Αν τα credentials είναι σωστά, επιστρέφει access_token και token_type: "bearer". Αν είναι λάθος, επιστρέφει 401 Unauthorized.

- POST /register

Δημιουργεί νέο χρήστη. Ελέγχει πρώτα αν υπάρχει ήδη χρήστης με το ίδιο email και αν υπάρχει επιστρέφει 409 Conflict. Αν το email είναι διαθέσιμο, κάνει hash το password, δημιουργεί νέο User και τον αποθηκεύει στη βάση. Αν το email είναι το FIRST_SUPERUSER, ο χρήστης παίρνει αυτόματα admin δικαιώματα.

- GET /me

Επιστρέφει τα στοιχεία του τρέχοντος authenticated user. Το endpoint απαιτεί authentication, γιατί παίρνει τον χρήστη από το dependency CurrentUser. Αν ο χρήστης δεν είναι authenticated, επιστρέφεται 401 Unauthorized.

- POST /users/favorites

Προσθέτει μία ταινία στα αγαπημένα του τρέχοντος χρήστη. Πρώτα ελέγχει ότι ο χρήστης είναι authenticated και ότι η ταινία υπάρχει. Μετά ελέγχει αν η ταινία υπάρχει ήδη στα αγαπημένα. Επίσης δημιουργεί ή βρίσκει τον αντίστοιχο Rater για τον χρήστη και αποθηκεύει rating 5.0 για την ταινία. Αν το favorite υπάρχει ήδη, επιστρέφει την υπάρχουσα εγγραφή. Αν δεν υπάρχει, δημιουργεί νέο UserMovieFavorite. Εδώ έχει είναι λίγο μπερδεμα ο κωδικας γιατι αφορα την recommendation λογικη των ratings καθως απο το dataset που βρηκα υπηρχαν users με IDs που ειχαν βαθμολογισει τις monies ασχετοι με τους users της εφαρμογης για αυτο εφτιαξα εναν Rater που αντιστοιχει σε καθε user της εφαρμογης ετσι ωστε να μην μπερδευονται τα ids των εξωτερικων users με αυτα της εφαρμογης. στην αρχη απλα προσθετα +1 στο max του id αλλα αλλαξα στρατιγικη αλλα εχει περασει καιρος απο τοτε που την ειχα αφησει την εφαρμογη και δεν εχει ολοκληρωθει η λογικη αυτη ακομμα με το recomendation ικανοποιητικα

- GET /users/favorites

Επιστρέφει τις αγαπημένες ταινίες του τρέχοντος χρήστη. Υποστηρίζει pagination με skip και limit, καθώς και προαιρετικό search με q πάνω στον τίτλο της ταινίας. Το endpoint κάνει join ανάμεσα σε Movie και UserMovieFavorite ώστε να φέρει μόνο τις ταινίες που ανήκουν στα favorites του συγκεκριμένου χρήστη.

- DELETE /users/favorites/{movie_id}

Αφαιρεί μία ταινία από τα αγαπημένα του τρέχοντος χρήστη. Ελέγχει ότι ο χρήστης είναι authenticated και μετά ψάχνει το favorite με σύνθετο κλειδί (user_id, movie_id). Αν δεν βρεθεί επιστρέφει 404 Favorite movie not found. Αν βρεθεί το διαγράφει και επιστρέφει 204 No Content.

- GET /users/recommendations

Επιστρέφει τις αποθηκευμένες προτεινόμενες ταινίες για τον τρέχοντα χρήστη. Κάνει query στον πίνακα UserMovieRecommendation, κάνει join με τον πίνακα Movie, ταξινομεί τα αποτελέσματα με βάση το rank και επιστρέφει για κάθε πρόταση στοιχεία της ταινίας μαζί με score και rank. Υποστηρίζει pagination με skip και limit.

- GET /users/

Επιστρέφει λίστα χρηστών και είναι admin endpoint αφού χρησιμοποιεί dependency CurrentAdmin ως authority. Υποστηρίζει pagination με skip και limit, μετράει το συνολικό πλήθος των users και επιστρέφει τα αποτελέσματα ταξινομημένα κατά created_at σε φθίνουσα σειρά.

- PATCH /users/me

Επιτρέπει στον τρέχοντα χρήστη να ενημερώσει τα δικά του στοιχεία. Αν δοθεί νέο email, ελέγχει αν υπάρχει ήδη άλλος χρήστης με το ίδιο email και σε αυτή την περίπτωση επιστρέφει 409 Conflict. Έπειτα ενημερώνει τα πεδία που στάλθηκαν στο request και κάνει commit στη βάση.

- GET /users/me

Επιστρέφει τα στοιχεία του τρέχοντος χρήστη. Είναι protected endpoint γιατί βασίζεται στο CurrentUser dependency. Χρησιμοποιείται από το frontend για να γνωρίζει ποιος χρήστης είναι συνδεδεμένος.

Helper function movie_to_public(movie)

Δεν είναι endpoint, αλλά helper function. Μετατρέπει ένα Movie database object σε MoviePublic response schema. Περιλαμβάνει βασικά στοιχεία της ταινίας, όπως id, title, year, director, plot, poster_url, imdb_id, tmdb_id, timestamps και λίστα από genres. Χρησιμοποιείται κυρίως όταν επιστρέφονται αγαπημένες ταινίες.

APIRouter(prefix="/movies", tags=["movies"])

- GET /movies

Επιστρέφει λίστα ταινιών από τη βάση. Υποστηρίζει pagination με skip και limit καθώς και προαιρετική αναζήτηση με q πάνω στον τίτλο της ταινίας. Αν δοθεί q γίνεται LIKE query στο Movie.title δηλαδή αν υπάρχει το keyword της αναζήτησης οπουδήποτε μέσα στον τίτλο. Το response περιλαμβάνει λίστα από MoviePublic αντικείμενα και συνολικό count.

Χρησιμοποιείται για την κύρια προβολή ταινιών στο UI.

Admin movie endpoints

- GET /movies/admin/{movie_id}

Επιστρέφει αναλυτικά στοιχεία μίας ταινίας με βάση το movie_id. Είναι admin endpoint, επειδή καλεί ensure_admin(current_user). Αν η ταινία δεν υπάρχει, επιστρέφει 404 Movie not found.

- GET /movies/admin/{movie_id}/plot

Επιστρέφει μόνο το plot συγκεκριμένης ταινίας. Και αυτό απαιτεί admin δικαιώματα. Αν δεν βρεθεί η ταινία, επιστρέφει 404 Movie not found. Το response περιλαμβάνει id, title και plot.

- POST /movies/admin

Δημιουργεί νέα ταινία. Είναι admin endpoint και δέχεται MovieCreate schema. Αρχικά δημιουργεί Movie object μετα συνδέει τα genres με βάση τα genre_ids ενημερώνει το updated_at και μετά αποθηκεύει την ταινία στη MySQL. Παράλληλα προσθέτει την ταινία και στη Chroma vectorstore με movie_to_chroma_text() και movie_to_chroma_metadata().

Αν αποτύχει κάτι, γίνεται rollback στη βάση. Αν είχε ήδη προστεθεί κάτι στη Chroma, γίνεται προσπάθεια να διαγραφεί ώστε να μη μείνει ασυνέπεια ανάμεσα στη MySQL και το vectorstore.

- DELETE /movies/admin/{movie_id}

Διαγράφει μία ταινία με το id. Απαιτεί admin δικαιώματα. Πρώτα βρίσκει την ταινία στη βάση και αν δεν υπάρχει επιστρέφει 404 Movie not found. Πριν τη διαγράψει από τη MySQL, προσπαθεί να διαγράψει από τη Chroma όλα τα documents που έχουν movie_id ίσο με το id της ταινίας.

Αν η διαγραφή από Chroma αποτύχει, δεν διαγράφει την ταινία από τη MySQL και επιστρέφει 500, ώστε να μη μείνει η εφαρμογή σε ασυνεπή κατάσταση. Αν η Chroma διαγραφή πετύχει, τότε διαγράφει και την ταινία από τη βάση.

- GET /movies/admin/analytics/summary

Επιστρέφει συνοπτικά στατιστικά χρήσης του AI search. Υπολογίζει συνολικό αριθμό requests, το usage input tokens, output tokens και total tokens και συνολικό estimated cost από τον πίνακα SearchHistory. Μπορείς να βάλεις προαιρετικά φίλτρα ημερομηνίας date_from και date_to.

Χρησιμοποιείται για γενική εικόνα κόστους και χρήσης.

- GET /movies/admin/analytics/daily

Επιστρέφει στατιστικά χρήσης ανά ημέρα. Ομαδοποιεί τα SearchHistory records με βάση την ημερομηνία δημιουργίας και επιστρέφει για κάθε μέρα τον αριθμό requests, usage tokens και cost. Υποστηρίζει επίσης φίλτρα date_from και date_to.

- GET /movies/admin/analytics/sessions

Επιστρέφει usage cost analytics ανά search session. Κάνει join ανάμεσα σε SearchSession και SearchHistory, ομαδοποιεί ανά session και υπολογίζει requests τα tokens και κόστος για κάθε session. Τα αποτελέσματα ταξινομούνται με βάση το κόστος σε φθίνουσα σειρά και έχει pagination με skip και limit. Χρησιμοποιείται για να εντοπίζονται τα πιο ακριβά ή πιο ενεργά sessions.

- Helper functions ensure_admin(current_user)

Δεν είναι endpoint, αλλά χρησιμοποιείται από τα admin endpoints. Ελέγχει αν ο χρήστης είναι superuser. Αν δεν είναι, επιστρέφει 403 Forbidden με μήνυμα Admin privileges required.

- movie_to_chroma_text(movie)

Μετατρέπει μία ταινία σε κείμενο που θα αποθηκευτεί στη Chroma. Στην τρέχουσα υλοποίηση επιστρέφει το movie.plot ή κενό string αν δεν υπάρχει plot.

- movie_to_chroma_metadata(movie)

Δημιουργεί metadata για τη Chroma. Περιλαμβάνει movie_id, title, year, director, doc_type: "summary", genres ως string καθώς και boolean genre flags τύπου genre_action=True ώστε να γίνεται το filtering στην chroma. Αυτά τα metadata βοηθούν αργότερα στο semantic retrieval και στο filtering.

MODEL

User, Search Session και Agent Conversation Models

database models και API schemas που χρησιμοποιούνται για τη διαχείριση χρηστών, το authentication, τα agent chat sessions και την αποθήκευση του ιστορικού αναζητήσεων. Τα models υλοποιούνται με **SQLModel**, συνδυάζοντας ORM δυνατότητες τύπου SQLAlchemy με Pydantic-style validation schemas.

User Model

Κάθε χρήστης έχει βασικά πεδία authentication και profile, όπως:

- `email`
- `hashed_password`
- `full_name`
- `is_active`
- `is_superuser`
- `created_at`

`email` unique και indexed ώστε να μπορεί να χρησιμοποιηθεί αποδοτικά για login και αναζήτηση χρήστη. Το password δεν αποθηκεύεται σε απλή μορφή, αλλά ως `hashed_password`.

`is_superuser` χρησιμοποιείται για να διαχωρίζει τους απλούς χρήστες από τους administrators. Οι admin users μπορούν να έχουν πρόσβαση σε protected administrative endpoints, όπως user listing, movie management και analytics.

ο User ορίζει επίσης relationships με άλλες οντότητες:

- `favorite_movies`: συνδέει τον χρήστη με τις ταινίες που έχει προσθέσει στα αγαπημένα.
- `search_sessions`: συνδέει τον χρήστη με τα AI search chat sessions του.
- `rater`: συνδέει τον χρήστη με ένα rating profile.

Ενας χρήστης μπορεί να έχει πολλές αγαπημένες ταινίες, πολλά search sessions και ένα συνδεδεμένο rater profile.

UserBase

Το `UserBase` περιέχει κοινά πεδία του χρήστη, τα οποία μπορούν να χρησιμοποιηθούν από άλλα user-related schemas.

- `email`
- `is_active`
- `is_superuser`
- `full_name`

Αυτή η κλάση δεν αντιστοιχεί σε database table. Χρησιμοποιείται κυρίως ως reusable base schema για validation και API data structures.

SearchSession Model

Το `SearchSession` model αναπαριστά μία συνομιλία ανάμεσα σε έναν χρήστη και τον AI movie search agent.

Αντιστοιχεί στον πίνακα `search_session`.

Κάθε `search session` ανήκει σε έναν χρήστη μέσω του `owner_id` foreign key. Έτσι η εφαρμογή μπορεί να κρατά ξεχωριστές τις συνομιλίες κάθε χρήστη.

Βασικά στοιχεία:

- `id`: UUID primary key του session.
- `title`: τίτλος του session.
- `owner_id`: ο χρήστης που κατέχει το session.
- `memory`: η αποθηκευμένη μνήμη της συνομιλίας αφορά τις ερωτοαπαντήσεις για το history.
- `created_at`: timestamp δημιουργίας.
- `updated_at`: timestamp τελευταίας ενημέρωσης.

Το πεδίο `memory` αποθηκεύεται ως `LONGTEXT` και περιέχει serialized text σε μορφή JSON-like λίστας μηνυμάτων, για παράδειγμα:

```
[
  {"role": "user", "content": "Find me a dark sci-fi movie"},
  {"role": "assistant", "content": "You may like Blade Runner..."}
]
```

Σημαντικό είναι ότι το `memory` δεν είναι JSON column στη βάση, αλλά text field που αποθηκεύει serialized περιεχόμενο. Αυτή η μνήμη χρησιμοποιείται από τον agent για να καταλαβαίνει ερωτήσεις και να διατηρεί history context μέσα στην ίδια συνομιλία.

Το model ορίζει δύο βασικά relationships:

- `owner`: ο χρήστης στον οποίο ανήκει το session.
- `history`: όλες οι `SearchHistory` εγγραφές που δημιουργήθηκαν μέσα σε αυτό το session.

Άρα ένα `SearchSession` μπορεί να περιέχει πολλά `SearchHistory` records. Δηλαδή μια συνομιλία έχει πολλές ερωτοαπαντήσεις

SearchSessionBase

Το `SearchSessionBase` είναι abstract schema που περιέχει τα κοινά πεδία ενός search session.

Προς το παρόν περιέχει μόνο:

```
title: str
```

Το title έχει default τιμή πάντα αυτή, στην συγκεκριμένη υλοποίηση δεν δίνεται η δυνατότητα μέσω του UI να επιλέξει ονομα αλλά υπάρχει αυτή η δυνατότητα από θέμα backend μπαινει πάντα με την default ίδια τιμή `Untitled Search`.

SearchSessionCreate

Το **SearchSessionCreate** είναι το request schema που χρησιμοποιείται για δημιουργία νέου search session.

Κληρονομεί από το **SearchSessionBase**, οπότε μπορεί να δεχτεί τίτλο για το session. Το **owner_id** δεν δίνεται από τον client, αλλά προκύπτει στο backend από τον authenticated user.

SearchSessionPublic

Το **SearchSessionPublic** είναι response schema που επιστρέφεται στον client όταν ζητούνται πληροφορίες για ένα search session.

Επιστρέφει:

- **id**
- **title**
- **owner_id**
- **created_at**
- **updated_at**

Δεν εκθέτει το πλήρες εσωτερικό **memory**, ώστε το API response να παραμένει πιο καθαρό και ελεγχόμενο.

SearchSessionsPublic

Το **SearchSessionsPublic** είναι wrapper response schema για επιστροφή πολλών search sessions.

Έχει:

- **data**: λίστα από search sessions.
- **count**: συνολικό πλήθος sessions.

Αυτή η μορφή είναι χρήσιμη για paginated API responses.

SearchHistory Model

Το **SearchHistory** model αναπαριστά μία μεμονωμένη αλληλεπίδραση ανάμεσα στον χρήστη και τον AI agent μέσα σε συγκεκριμένο session.

Αντιστοιχεί στον πίνακα **search_history**.

Μία εγγραφή στο **SearchHistory** αντιστοιχεί συνήθως σε:

```
one user message -> one graph execution -> one agent response
```

Δηλαδή μια εγγραφή είναι μια ερώτηση του χρήστη με την απάντηση του AI.

Τα βασικά πεδία είναι:

- `id`: UUID primary key.
- `session_id`: το session στο οποίο ανήκει το μήνυμα.
- `owner_id`: ο χρήστης που έκανε το query.
- `query`: το μήνυμα που έστειλε ο χρήστης.
- `result`: η απάντηση που παρήγαγε ο agent.
- `input_tokens`: αριθμός input tokens που χρησιμοποιήθηκαν.
- `output_tokens`: αριθμός output tokens που δημιουργήθηκαν.
- `total_tokens`: συνολική χρήση tokens.
- `estimated_cost`: εκτιμώμενο κόστος εκτέλεσης του agent.
- `created_at`: timestamp της αλληλεπίδρασης.

Το πεδίο `result` αποθηκεύεται ως `LONGTEXT`, επειδή οι απαντήσεις του agent μπορεί να είναι μεγάλες.

Το `estimated_cost` χρησιμοποιεί `Decimal` και `Numeric(12, 6)`, κάτι που είναι πιο κατάλληλο από `float` για αποθήκευση τιμών κόστους και ενημερώνεται σε κάθε βήμα του βρογχου που θα χρησιμοποιηθεί κάποιο LLM όπως πχ για το `rewrite query` ή την τελική απάντηση.

SearchHistoryBase

Το `SearchHistoryBase` περιέχει τα κοινά πεδία για μία εγγραφή ιστορικού.

Περιλαμβάνει:

- `query`
- `result`
- `input_tokens`
- `output_tokens`
- `total_tokens`
- `estimated_cost`

Αυτό το base schema επαναχρησιμοποιείται τόσο από το database model όσο και από τα public API schemas στο router.

SearchHistoryCreate

Το `SearchHistoryCreate` είναι schema για δημιουργία νέας εγγραφής ιστορικού.

Κληρονομεί τα βασικά πεδία `query` `result` `token usage` και προσθέτει:

```
session_id: uuid.UUID
```

SearchHistoryPublic

Το `SearchHistoryPublic` είναι το response schema που χρησιμοποιείται όταν επιστρέφεται μία εγγραφή ιστορικού στο frontend.

Επιστρέφει:

- `id`
- `session_id`
- `owner_id`
- `query`
- `result`
- token usage
- estimated cost
- `created_at`

Έτσι το frontend μπορεί να εμφανίσει προηγούμενα μηνύματα και αν χρειάζεται πληροφορίες για usage και cost.

SearchHistoriesPublic

Το `SearchHistoriesPublic` είναι wrapper schema για επιστροφή πολλών history entries.

Έχει:

- `data`: λίστα από history records.
 - `count`: συνολικό πλήθος records.
-

SearchSessionWithHistoryPublic

Το `SearchSessionWithHistoryPublic` επεκτείνει το `SearchSessionPublic`, προσθέτοντας και τη λίστα των history entries του συγκεκριμένου session.

Είναι χρήσιμο όταν το frontend χρειάζεται να φορτώσει μία πλήρη συνομιλία, δηλαδή και τα metadata του session και όλα τα προηγούμενα user-agent ερωτοαπαντήσεις.

AgentChatRequest

Το `AgentChatRequest` είναι το request schema για αποστολή μηνύματος στον AI agent.

Περιέχει:

```
message: str
```

Το frontend στέλνει αυτό το schema στο chat endpoint όταν ο χρήστης γράφει νέο μήνυμα.

Παράδειγμα request:

```
{  
  "message": "Find me a dark science fiction movie with philosophical themes"
```

```
}
```

AgentChatResponse

Το **AgentChatResponse** είναι το response schema που επιστρέφεται από το agent chat endpoint.

Περιέχει:

- **session_id**: το session στο οποίο ανήκει η συνομιλία.
- **reply**: η τελική απάντηση που παρήγαγε ο agent.

Παράδειγμα response:

```
{
  "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "reply": "You may like Blade Runner because it explores identity, memory, and
artificial intelligence."
}
```

Authentication Helper Schemas

Το αρχείο ορίζει επίσης μερικά γενικά schemas που χρησιμοποιούνται από το authentication σύστημα.

Message

Το **Message** είναι ένα απλό response schema για επιστροφή μηνυμάτων από το API.

Παράδειγμα:

```
{
  "message": "User updated successfully"
}
```

Token

Το **Token** αναπαριστά το authentication token που επιστρέφεται μετά το login.

Περιέχει:

- **access_token**
- **token_type**

Το default **token_type** είναι "bearer".

TokenPayload

To **TokenPayload** αναπαριστά το decoded JWT payload.

NewPassword

To **NewPassword** χρησιμοποιείται σε password reset flows.

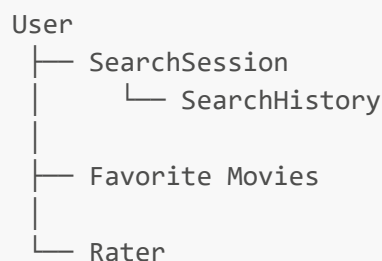
Περιέχει:

- **token**
- **new_password**

Το νέο password έχει validation constraints για ελάχιστο και μέγιστο μήκος.

Relationship Overview

Οι βασικές σχέσεις ανάμεσα στα models είναι:



Πιο συγκεκριμένα:

- Ένας χρήστης μπορεί να έχει πολλά search sessions.
- Ένα search session μπορεί να έχει πολλά search history entries.
- Κάθε search history entry ανήκει σε ένα session και σε έναν user.
- Ένας χρήστης μπορεί να έχει πολλές favorite movies.
- Ένας χρήστης μπορεί να συνδέεται με ένα rater profile.

Στο **User** model υπάρχει relationship προς **search_sessions**, αλλά όχι ξεχωριστό relationship τύπου **search_history**. Η σύνδεση με το ιστορικό γίνεται κυρίως μέσω **SearchSession.history** και μέσω του **owner_id** στο **SearchHistory**.

Το **User** model διαχειρίζεται το authentication και την ιδιοκτησία των user data. Το **SearchSession** model αναπαριστά μία συνομιλία με τον AI movie agent. Το **SearchHistory** model αποθηκεύει κάθε user query και generated response, μαζί με token usage και estimated cost. Τα request και response schemas ορίζουν καθαρά API contracts για session creation, history retrieval, authentication και agent chat interaction.

Στο frontend χρησιμοποιήσα Angular standalone components δλδ κάθε component δηλώνει μόνο του τι χρειάζεται. Η πλοήγηση γίνεται με το Angular Router. Έχουμε δύο βασικές περιοχές στην εφαρμογή το user area και το admin area. Το user area βρίσκεται κάτω από το /user και περιλαμβάνει τις σελίδες movies, ask-ai, recommendations και favorites. Το admin area βρίσκεται κάτω από το /admin και περιλαμβάνει dashboard, movie management, movie ingestion και analytics. Και στις δύο περιπτώσεις χρησιμοποιήσαμε

nested routing, δηλαδή ένα layout component κρατάει σταθερό το sidebar/menu και μέσα του αλλάζει το περιεχόμενο μέσω router-outlet.

Ένα από τα πιο σημαντικά κομμάτια είναι οι guards. Ο authGuard προστατεύει όλο το user section. Δηλαδή αν κάποιος προσπαθήσει να μπει σε /user/movies, /user/ask-ai, /user/favorites ή /user/recommendations χωρίς να είναι συνδεδεμένος, ο guard τον στέλνει στο /login. Ουσιαστικά προστατεύει τις σελίδες που αφορούν προσωπικά δεδομένα χρήστη, όπως favorites, συζητήσεις με το AI και recommendations. Δεν αφήνει έναν μη authenticated χρήστη να δει ή να χρησιμοποιήσει αυτές τις λειτουργίες.

Ο adminGuard είναι πιο αυστηρός. Πρώτα ελέγχει αν ο χρήστης είναι logged in και μετά ελέγχει αν είναι superuser. Αν δεν είναι logged in, τον στέλνει στο login. Αν είναι logged in αλλά δεν είναι admin, δεν τον αφήνει να μπει στο /admin. Αυτό προστατεύει λειτουργίες όπως διαγραφή ταινιών, προσθήκη/ingestion νέας ταινίας στη MySQL και στη ChromaDB, καθώς και τα analytics κόστους και usage. Η αυθεντικοποίηση γίνεται με JWT authentication. Μετά το login το backend επιστρέφει ένα access_token, το οποίο αποθηκεύεται στο localStorage. Μέσα στο JWT payload έχουμε πληροφορίες όπως το sub, το exp και το is_superuser. Το frontend κάνει decode το token για να καταλάβει αν ο χρήστης είναι απλός user ή admin. Αυτό χρησιμοποιείται για να γίνει σωστό redirect μετά το login: ο απλός χρήστης πηγαίνει στο user panel, ενώ ο admin πηγαίνει στο admin panel. Όμως αυτός ο έλεγχος στο frontend χρησιμεύει κυρίως για σωστή εμπειρία χρήστη. Η πραγματική ασφάλεια πρέπει πάντα να γίνεται και στο backend, με ελέγχους όπως current_user.is_superuser.

Για τα HTTP requests χρησιμοποιήσαμε Angular HttpClient και HTTP interceptor. Ο interceptor προσθέτει αυτόματα σε κάθε request το header. Authorization με Bearer token. Έτσι δεν χρειάζεται κάθε service να γράφει χειροκίνητα το token. Όταν το frontend κάνει request σε endpoints όπως /movies, /users/favorites, /users/recommendations, /searches ή /admin/movies, το JWT στέλνεται αυτόματα.

Επίσης χρησιμοποιώ services layers στο Angular για να διαχωρίσουμε την επικοινωνία με το backend από το UI. Για παράδειγμα, το MovieService μιλάει με τα endpoints για ταινίες, favorites και recommendations. Το SearchService μιλάει με τα endpoints του RAG chat και των search sessions. Το AdminService μιλάει με τα admin endpoints για movies, ingestion και analytics. Έτσι τα components δεν χρειάζεται να ξέρουν λεπτομέρειες των URLs ή των HTTP calls. Τα components ασχολούνται κυρίως με το τι εμφανίζεται, αν φορτώνει κάτι, αν υπάρχει error, και τι γίνεται όταν ο χρήστης πατήσει κάποιο κουμπί.

Στο admin κομμάτι υπάρχει δυνατότητα διαχείρισης ταινιών, προσθήκης νέας ταινίας και ingestion. Όταν ο admin προσθέτει μια ταινία, αυτή αποθηκεύεται στη MySQL και παράλληλα γίνεται ingest στη ChromaDB. Τα genres δεν χρησιμοποιούνται μόνο ως απλή λίστα, αλλά μετατρέπονται και σε metadata flags, όπως genre_action = true ή genre_comedy = true, ώστε αργότερα να μπορούν να χρησιμοποιηθούν σε filtering μέσα στο vector search. Το admin έχει επίσης analytics για usage, tokens και εκτιμώμενο κόστος.

Η εφαρμογή ακολουθεί μια full-stack αρχιτεκτονική. Το Angular χειρίζεται το UI, το routing, τα guards και την επικοινωνία με το API. Το FastAPI χειρίζεται authentication, authorization, business logic, database access και RAG logic. Η MySQL κρατάει τα κανονικά relational δεδομένα, ενώ η ChromaDB κρατάει τα vector embeddings για semantic search. Οι guards, τα services, το interceptor και τα layouts βοηθούν ώστε η εφαρμογή να είναι μια οργανωμένη εφαρμογή με ρόλους προστατευμένες περιοχές και separation of concerns.